

I'll provide a comprehensive explanation of trees and their implementations, building from fundamental concepts to more complex operations.

LECTURE PART

1. Introduction to Trees

A tree is a hierarchical data structure that consists of nodes connected by edges. Unlike linear data structures like arrays or linked lists, trees represent hierarchical relationships between elements. Think of a family tree or an organizational chart - they both demonstrate the hierarchical nature of tree structures.

```
```java
class TreeNode<T> {
 T data;
 List<TreeNode<T>> children;

 public TreeNode(T data) {
 this.data = data;
 this.children = new ArrayList<>();
 }
}
```
```

2. Trees and Terminology

Let's understand the key terms used in tree data structures:

```
```java
class BinaryTreeNode<T> {
 T data; // The value stored in the node
 BinaryTreeNode<T> left; // Reference to left child
 BinaryTreeNode<T> right; // Reference to right child
 BinaryTreeNode<T> parent; // Reference to parent node (optional)

 public BinaryTreeNode(T data) {
 this.data = data;
 this.left = null;
 this.right = null;
 this.parent = null;
 }
}
```
```

Key terminology:

- Root: The topmost node (the starting point)
- Parent: A node that has children
- Child: A node directly connected to another node when moving away from the root
- Leaf: A node with no children
- Height: The length of the longest path from root to leaf
- Depth: The length of the path from a node to the root
- Level: All nodes at the same depth form a level

3. Tree Traversals

There are several ways to traverse a tree, each serving different purposes:

```
```java
public class TreeTraversal<T> {
 // Depth-First Search Traversals
 public void inorder(BinaryTreeNode<T> root) {
 if (root == null) return;
 inorder(root.left); // Visit left subtree
 System.out.print(root.data); // Visit root
 inorder(root.right); // Visit right subtree
 }

 public void preorder(BinaryTreeNode<T> root) {
 if (root == null) return;
 System.out.print(root.data); // Visit root
 preorder(root.left); // Visit left subtree
 preorder(root.right); // Visit right subtree
 }

 public void postorder(BinaryTreeNode<T> root) {
 if (root == null) return;
 postorder(root.left); // Visit left subtree
 postorder(root.right); // Visit right subtree
 System.out.print(root.data); // Visit root
 }

 // Breadth-First Search (Level Order) Traversal
 public void levelOrder(BinaryTreeNode<T> root) {
 if (root == null) return;
 Queue<BinaryTreeNode<T>> queue = new LinkedList<>();
 queue.offer(root);

 while (!queue.isEmpty()) {
 BinaryTreeNode<T> current = queue.poll();
 System.out.print(current.data);

 if (current.left != null)
 queue.offer(current.left);
 if (current.right != null)
 queue.offer(current.right);
 }
 }
}
```
```

4. Complete Binary Trees and Almost Complete Binary Trees (ACBT)

A complete binary tree is a binary tree where all levels are filled except possibly the last level, which is filled from left to right. An almost complete binary tree follows the same rules but may have the last level partially filled.

5. Array Implementation of ACBT

We can represent a binary tree using an array, where for a node at index i :

- Left child is at index $2i + 1$
- Right child is at index $2i + 2$
- Parent is at index $(i-1)/2$

```
```java
public class ArrayBinaryTree<T> {
 private T[] tree;
 private int capacity;
 private int size;

 @SuppressWarnings("unchecked")
 public ArrayBinaryTree(int capacity) {
 this.capacity = capacity;
 this.tree = (T[]) new Object[capacity];
 this.size = 0;
 }

 public void insert(T data) {
 if (size >= capacity) {
 throw new IllegalStateException("Tree is full");
 }
 tree[size++] = data;
 }

 public T getLeft(int index) {
 int leftIndex = 2 * index + 1;
 return leftIndex < size ? tree[leftIndex] : null;
 }

 public T getRight(int index) {
 int rightIndex = 2 * index + 2;
 return rightIndex < size ? tree[rightIndex] : null;
 }
}
```
```

6. Binary Search Trees (BST)

A binary search tree is a binary tree where for each node:

- All nodes in the left subtree have values less than the node's value
- All nodes in the right subtree have values greater than the node's value

LAB IMPLEMENTATION

Here's a complete implementation of a Binary Search Tree with all the requested operations:

```

```java
public class BinarySearchTree<T extends Comparable<T>> {
 private BinaryTreeNode<T> root;

 // Create operation
 public void create() {
 this.root = null;
 }

 // Insert operation
 public void insert(T data) {
 root = insertRec(root, data);
 }

 private BinaryTreeNode<T> insertRec(BinaryTreeNode<T> root, T data) {
 if (root == null) {
 return new BinaryTreeNode<>(data);
 }

 int comparison = data.compareTo(root.data);
 if (comparison < 0) {
 root.left = insertRec(root.left, data);
 } else if (comparison > 0) {
 root.right = insertRec(root.right, data);
 }

 return root;
 }

 // Breadth First Search
 public void bfs() {
 if (root == null) return;
 Queue<BinaryTreeNode<T>> queue = new LinkedList<>();
 queue.offer(root);

 while (!queue.isEmpty()) {
 int levelSize = queue.size();
 for (int i = 0; i < levelSize; i++) {
 BinaryTreeNode<T> current = queue.poll();
 System.out.print(current.data + " ");

 if (current.left != null)
 queue.offer(current.left);
 if (current.right != null)
 queue.offer(current.right);
 }
 System.out.println(); // New line for each level
 }
 }

 // Depth First Search Traversals

```

```

public void inorder() {
 inorderRec(root);
 System.out.println();
}

private void inorderRec(BinaryTreeNode<T> root) {
 if (root == null) return;
 inorderRec(root.left);
 System.out.print(root.data + " ");
 inorderRec(root.right);
}

public void preorder() {
 preorderRec(root);
 System.out.println();
}

private void preorderRec(BinaryTreeNode<T> root) {
 if (root == null) return;
 System.out.print(root.data + " ");
 preorderRec(root.left);
 preorderRec(root.right);
}

public void postorder() {
 postorderRec(root);
 System.out.println();
}

private void postorderRec(BinaryTreeNode<T> root) {
 if (root == null) return;
 postorderRec(root.left);
 postorderRec(root.right);
 System.out.print(root.data + " ");
}

// Delete operation
public void delete(T data) {
 root = deleteRec(root, data);
}

private BinaryTreeNode<T> deleteRec(BinaryTreeNode<T> root, T data) {
 if (root == null) {
 return null;
 }

 int comparison = data.compareTo(root.data);
 if (comparison < 0) {
 root.left = deleteRec(root.left, data);
 } else if (comparison > 0) {
 root.right = deleteRec(root.right, data);
 } else {

```

```

 // Node with only one child or no child
 if (root.left == null) {
 return root.right;
 } else if (root.right == null) {
 return root.left;
 }

 // Node with two children: Get the inorder successor (smallest
 // in the right subtree)
 root.data = minValue(root.right);

 // Delete the inorder successor
 root.right = deleteRec(root.right, root.data);
 }

 return root;
}

private T minValue(BinaryTreeNode<T> root) {
 T minv = root.data;
 while (root.left != null) {
 minv = root.left.data;
 root = root.left;
 }
 return minv;
}

// Example usage
public static void main(String[] args) {
 BinarySearchTree<Integer> bst = new BinarySearchTree<>();
 bst.create();

 // Insert some values
 bst.insert(50);
 bst.insert(30);
 bst.insert(70);
 bst.insert(20);
 bst.insert(40);
 bst.insert(60);
 bst.insert(80);

 System.out.println("BFS Traversal:");
 bst.bfs();

 System.out.println("\nInorder Traversal:");
 bst.inorder();

 System.out.println("\nPreorder Traversal:");
 bst.preorder();

 System.out.println("\nPostorder Traversal:");
 bst.postorder();
}

```

```
// Delete a node
bst.delete(30);
System.out.println("\nAfter deleting 30:");
bst.inorder();
 }
}
```

This implementation provides a complete set of operations for a binary search tree, including creation, various traversal methods, and deletion. The code is well-structured and includes comments to explain the functionality of each operation.

Would you like me to explain any specific part in more detail or provide additional examples of tree operations?